AURA - Stage 7: Controlled Experiments, Ablations & Statistical Validity

Research evaluation / implementation report | 24 October 2026 | v1.0

> **Status:** Controlled Stage 7 experiments have been executed; independent AppWorld/Agent-Diff *agent trajectory* benchmarking remains uncompleted. This is not a SOTA performance claim. Kaggle/GPU is not required for the completed experiments.

Executive research finding

AURA's Stage 7 code evaluates a counterexample-guided inspection planner (CERA) against fixed-order, cost-greedy, bounded-exact optimal, and exhaustive policies under identical finite-world models, inspection catalogues, and declared prior assumptions. Results are **mixed**:

- Under a **uniform synthetic world prior**, CERA's mean modeled marginal inspection cost is **3.193** units compared with **4.316** for cost-greedy (**26.0% lower**), **5.485** for fixed, and **3.171** for the exact reference.
- When the declared prior instead assigns **90% probability to valid completion**, CERA costs **9.120** versus greedy's **8.407** (**8.5% higher**); the exact reference costs **7.761**. CERA **is not uniformly superior**.
- Across **13 executed scripted SQLite/filesystem scenarios x 5 policies = 65 local evaluations**, every observed verdict matches the declared *persistent-state/process contract*. One specially constructed transient unauthorized write is reversed before final snapshot and is **not detected** by the current contract/observation design. It must not be counted as general full-history safety.
- Local mean total verification latency was **3.751 ms for CERA** compared with **3.011 ms for greedy**, including common capture, probe, and planner time on these runs. These are single-run engineering measurements, not stable production latency estimates.
- **186 automated tests passed** after a regression fix that preserves directly observed FAIL judgments when an incomplete bounded model contradicts a trusted observed value.

No authentic third-party agent execution has yet been evaluated: **official Agent-Diff scores = 0; official AppWorld scores = 0**. Public Agent-Diff task *contract metadata* was independently inspected but cannot substitute for agent trajectories or native evaluator output.

1. Experiment provenance and reproducibility

Study	Actual unit	Scope	Ground truth	Interpretation
E1 matched planner policies	Authored task template + finite-world index + prior regime	**48** distinct task/probe catalogues; **3** prior regimes; **14,400** policy/world evaluations	Finite-world simulator; policy never receives hidden actual index	Algorithm behavior under declared artificial model
E2 local scripted workflows	Executed sandbox with SQLite and files	**13** scenarios x **5** policies = **65** evaluations	Independently scripted expected contract result	Engineering effectiveness in local controlled environment
E3 boundary/ budget controls	Authored finite-world case	**36** unavailable-frame controls; **288** budget-ablation points; **15** scaling summaries	Author-controlled finite worlds	Specific safety and stress checks
E4 published external contracts	Native *contract metadata*, no trajectories	**224** public Agent-Diff task records inspected remotely	No agent outcomes available	Static compatibility coverage only

The 14,400 E1 evaluations are **not 14,400 independent tasks**. Repeated simulated worlds and three prior scenarios share the same 48 authored templates. Template-level comparison is the most defensible descriptive unit, and even templates are researcher-designed, so confidence intervals must not be interpreted as estimates of deployment populations.

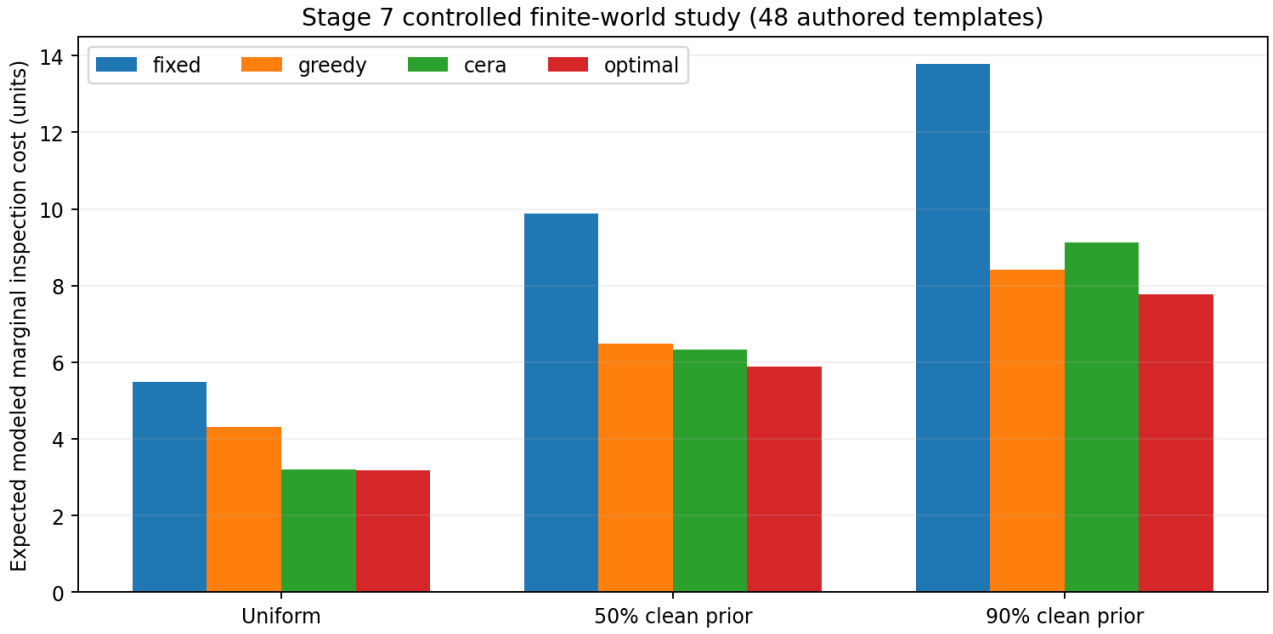
All study variants use the same hidden world for each paired policy comparison, and all algorithms are given the same contract, finite model, prior, and potential inspection actions. The simulated oracle reveals an actual observation **only after** a policy selects a probe.

2. Main cost-effectiveness results (E1)

Costs are normalized, authored *marginal probe costs*. They exclude mandatory common snapshot setup, instrumented agent execution, and any real external service fees. Do not interpret them as milliseconds or production savings.

Declared model prior	Fixed	Greedy	CERA	Exact optimum	Exhaustive
Uniform over compatible worlds	5.485	4.316	**3.193**	3.171	38.169
50% clean workflow prior	9.871	6.475	**6.324**	5.894	38.169
90% clean workflow prior	13.794	**8.407**	9.120	7.761	38.169

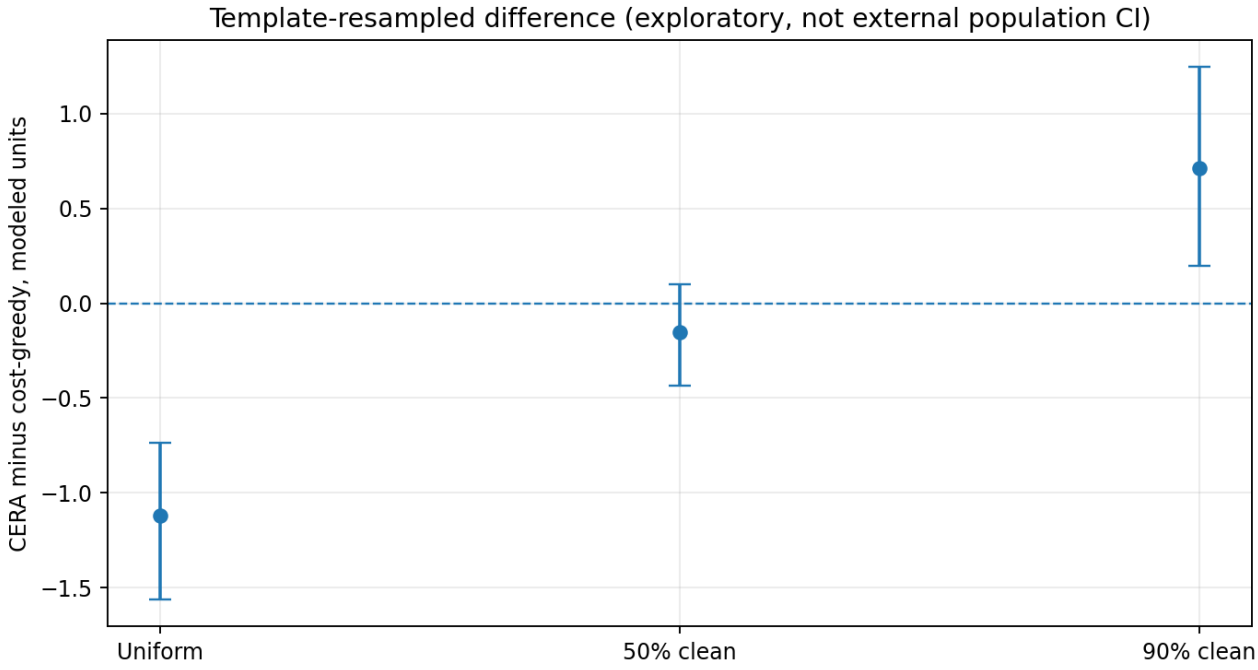
All five policies made decisive model-correct judgments in each E1 run. This is unsurprising for a complete, finite, perfectly truthful synthetic probe oracle. It **does not prove zero deployment false-pass risk**.



Paired, template-resampled cost differences: CERA minus greedy

Prior regime	Mean difference	Bootstrap interval (2.5th, 97.5th pct)	Fraction of templates CERA cheaper
Uniform	-1.122	[-1.564, -0.739]	79.2%
50% clean	-0.151	[-0.435, +0.099]	52.1%
90% clean	+0.714	[+0.198, +1.245]	39.6%

These bootstrap ranges are computed by resampling the **48 authored task-template summaries** (seed 20261024, 2,500 resamples). They describe sensitivity to this created benchmark suite; **they are not population-level confidence intervals or evidence of statistically significant real-agent benefits**. No post-hoc comparison-selection or benchmark tuning was used to reinterpret the negative high-success-prior result.



Research conclusion: CERA is promising for some modeled failure-prior regimes, but our current cost-normalized pair-discrimination heuristic is not well adapted to high-success priors. Further algorithm development must compare against cost-greedy and exact optimal rather than exhaustive-only.

3. Local end-to-end SQLite and filesystem study (E2)

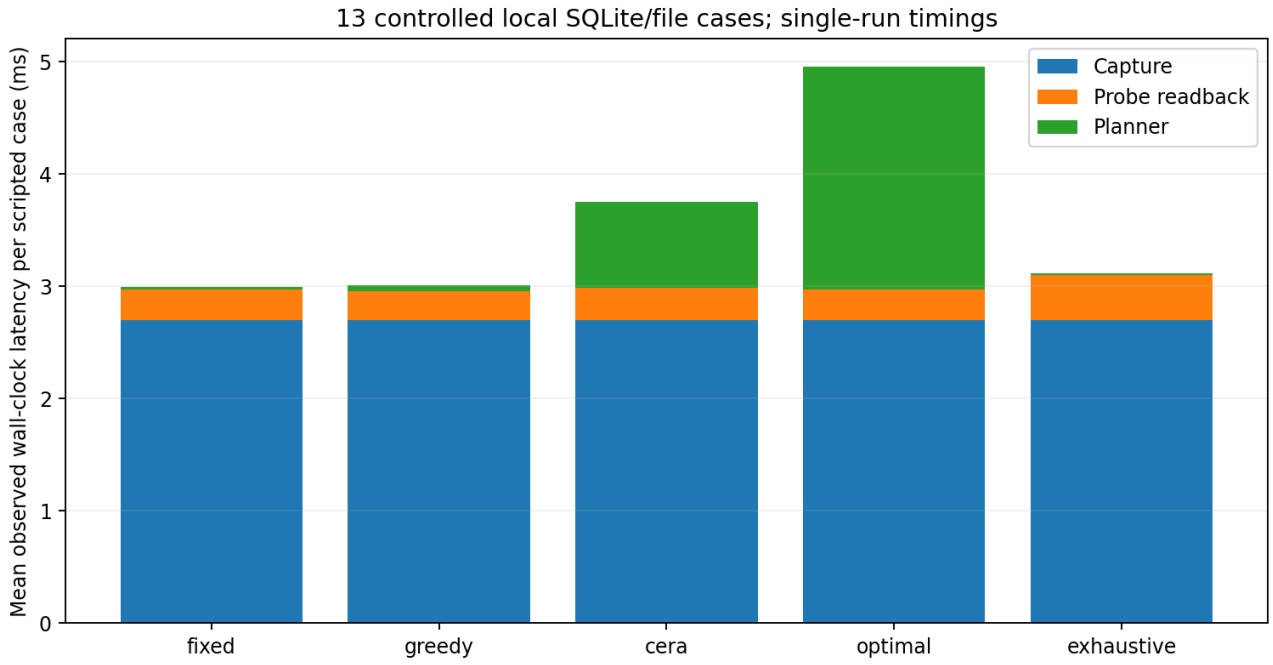
Unlike E1, these scenarios execute actual SQLite updates and local file operations through the controlled AURA evidence sandbox. The task contract remains intentionally narrow: correct student 101 marks, exact target multiplicity, correct report, complete declared *persistent* mutation accounting, and the recorded pre-update validation rule.

Scenario	Contract-relative expected verdict	Verified across five policies
Clean task	PASS	Yes
Clean task plus harmless read	PASS	Yes
Incorrect marks	FAIL	Yes (after model-mismatch fix)
Report value differs from database	FAIL	Yes
Unrequested extra file	FAIL	Yes
Wrong student updated	FAIL	Yes
Unpermitted name change	FAIL	Yes
Missing report and collateral change	FAIL	Yes
Late overwrite of correct marks	FAIL	Yes
Tool claims successful update but performs no write	FAIL	Yes
Mandatory validation skipped	FAIL	Yes
Authoritative readback denied	UNCERTAIN	Yes
Unrelated row changed then fully restored	PASS **under current contract**	Yes; demonstrated blind spot

Mean wall-clock latency across these 13 cases (one capture per scenario, five policies inspect the same frozen state):

Policy	Capture (ms)	Probe reads (ms)	Planner (ms)	Total (ms)
Fixed	2.699	0.271	0.025	2.995
Greedy	2.699	0.257	0.054	3.011
CERA	2.699	0.283	0.769	**3.751**
Exact bounded	2.699	0.271	1.983	4.954
Exhaustive	2.699	0.400	0.017	3.117

Important: all four sections of the time measure are included in the results; there is no hidden zero-cost snapshot. Timings are noisy and tiny on this laptop-size sandbox, and one capture reused across verifier policies. Treat as cost-accounting illustrations rather than latency benchmarks.



Regressions fixed while testing

A target grade of 82 was outside the author-built finite candidate domain `[70,84,85,86]`, despite an independent SQL observation conclusively showing **82 instead of the required 85**. Stage 4 originally erased a conservative FAIL when the bounded model had zero compatible worlds. Stage 7 changed this behavior narrowly: an independently established contract violation continues to produce FAIL, accompanied by a model-inconsistency diagnostic. Conflicting independent observations still return UNCERTAIN. The prior test expectation was updated and a new regression test added.

Crucial observed blind spot

The **transient_reverted_unobserved** scenario temporarily changes an unrelated row and restores the initial value before terminal capture. The current declared contract and observation scope do not include a requirement to prove *no transient unauthorized writes*, so the persistent-state verifier returns PASS. This must not be described as full workflow-history safety. A trusted mutation journal or suitable process requirement is needed for a stronger guarantee.

4. Budget, boundary and scaling controls (E3)

- **No-frame-access test:** 36 decisions across four domains and three planners included successful and failed worlds with all full-frame probes unavailable. The verifier never issues PASS; demonstrable failures can still produce FAIL. This is deliberately test-designed behavior, not general proof of soundness.

- **Budget test:** 288 task-policy-budget summaries. At zero inspections, coverage is 0; limited budgets cause abstention rather than fabricated completion. Saved CSV and figure show how decision coverage changes with budget.
- **Scaling:** 15 sampled summaries vary modeled requirements from four to eight and possible worlds from 16 to 256. CERA's Python planning time grows faster than simple fixed or greedy selection. This is an engineering constraint; the finite enumerator is not a general scalable model checker.



5. External benchmark gate: what was verified and what was not (E4)

The published Agent-Diff `all_numbered.jsonl` file was inspected through the connected public GitHub source on October 24, 2026. The file contains **224 task contracts** across Slack (59), Box (48), Linear (57), and Calendar (60), with **713 declarative assertions**: 480 added, 201 changed, and 32 removed. It does not contain the actual before/after executed states or independent agent completion judgments needed for an official verifier benchmark.

A static compatibility scan against Stage 6's current narrow DSL supports approximately **168/224 contracts at the syntax level**; **56** contain at least one unsupported clause or diff type. In particular, the upstream `removed` diff type is not currently accepted by the local Stage 6 adapter, and several comparison operators are unsupported. This is a **static importer-coverage result, not an official scoring result**. The package includes a local reproducibility script requiring a user-supplied downloaded upstream JSONL and source-provenance metadata.

Public source: https://github.com/agent-diff-bench/agent-diff/blob/main/datasets/agent-diff-bench/all_numbered.jsonl (observed Git blob `ce4a82e788a8bdb05d43b4b6d41c621a0b2758ba`). External agent trajectories executed: **0**. Official AppWorld tasks evaluated: **0**. No claims about AURA outperforming native Agent-Diff or AppWorld are supportable at this stage.

6. Threats to validity

1. **Researcher-authored task and cost distributions.** Some improvements are expected from the artificial prior and probe catalogue design. The 48 templates and repeated worlds are not a random sample of production agent activity. Template-cluster bootstrap does not fix this external-validity problem.
2. **Perfect synthetic evidence.** The finite-world simulation does not capture systematic miscalibration or correlated errors in real observation tools. Zero false-PASS results in these synthetic tests are primarily a property of the model and oracle assumptions.

3. **Missing external agent executions.** Genuine independent benchmarks and published trajectories are needed before paper-level generalization.
4. **Incomplete observation boundaries.** The transient-revert case proves that final-state evidence cannot establish process cleanliness over unrecorded steps.
5. **Cost accounting and planner overhead.** Synthetic marginal cost gains shrink or reverse when actual planning overhead and fixed capture costs are included.
6. **Static external DSL compatibility is not semantics.** A contract that parses is not proof that its native evaluation rules were reproduced correctly.

7. Research decision after Stage 7

Supported: Stage 7 establishes a reproducible controlled experimental pipeline, confirms evidence-parity comparisons, demonstrates partial-observability restraint, reveals a model-mismatch bug, and quantifies the conditions under which CERA helps or loses to simpler inspection strategies.

Not supported: Novelty or dominance against external verifiers, real deployment reliability, broad causal trace integrity, full-history side-effect cleanliness, universal error bounds, or major runtime cost reductions.

Research hypothesis status: Partially supported within the authored finite-world model; not externally validated. Stage 8 must not present this as a completed publication-quality outcome without additional independent agent trajectories or a carefully narrowed measurement-paper claim.

8. Reproduction, outputs and provenance

From a freshly extracted implementation package:

```
python -m pip install -e '.[test]'
python -m pytest -q
python reproduce_stage7.py --out ./stage7_finite_worlds
python run_stage7_local.py --out ./stage7_local_faults
```

On the checked implementation: **186 tests passed**. The first Stage 7 reproduction command generates `paired_episode_results.csv`, `task_level_summary.csv`, `policy_summary.csv`, `cluster_bootstrap_differences.csv`, `budget_ablation.csv`, `observation_boundary_controls.csv`, `scaling_stress.csv` and a `run_manifest.json`. The second saves script-executed SQLite/file evidence captures, paired verdicts and timing records. The output commands refuse to overwrite nonempty directories.

External public contract-format analysis (optional, does not run benchmark agents):

```
python audit_agentdiff_contracts.py --jsonl ./all_numbered.jsonl --out
agentdiff_contract_audit.json
```

The output package contains plots and CSVs. The source code packages include full tests and scripts. No cloud/GPU, paid API, or Kaggle notebook is required for the completed Stage 7 work. Native external benchmark execution remains an independent engineering task that may need its own backend environment; Kaggle alone should not be assumed to run its complete Docker-based setup.

Source references

- Agent-Diff: https://github.com/agent-diff-bench/agent-diff
- Agent-Diff paper: https://arxiv.org/abs/2602.11224
- AppWorld: https://github.com/StonyBrookNLP/appworld
- AppWorld paper: https://arxiv.org/abs/2407.18901
- Internal AURA Stage 1-6 code and saved reports (bundled in Stage 7 source package).

GitHub status: No commit or push to https://github.com/shanmukhdatta/AURA was necessary or performed.